

```
In [5]: import numpy as np
        np.random.rand(1,10,2)
```

```
Out[5]: array([[0.84634626, 0.32543562],
               [0.93571295, 0.41249566],
               [0.06525547, 0.34950532],
               [0.38865339, 0.77078367],
               [0.04546726, 0.42140742],
               [0.83670652, 0.69903672],
               [0.88532073, 0.6520241 ],
               [0.45328327, 0.24292279],
               [0.53644992, 0.5907537 ],
               [0.97554882, 0.81137787]])
```

```
In [3]: import numpy as np
        np.random.randint(1,10,2)
```

```
Out[3]: array([4, 5])
```

```
In [4]: np.random.randint(21,10,2)
```

```
-----
ValueError                                Traceback (most recent call last)
Cell In[4], line 1
----> 1 np.random.randint(21,10,2)

File numpy\random\mtrand.pyx:780, in numpy.random.mtrand.RandomState.randint()

File numpy\random\_bounded_integers.pyx:1425, in numpy.random._bounded_integers._rand_int32()

ValueError: low >= high
```

```
In [7]: #arr1 = np.array([1,2,3,4,5])
        b=np.arange(0, 10, 3) # create array with space value
        print("Array of b:",b)
```

```
Array of b: [0 3 6 9]
```

```
In [8]: d = np.zeros((2,3)) # create a array filled with zeros
        print("Array of d:\n",d)
```

```
Array of d:
[[0. 0. 0.]
 [0. 0. 0.]
```

```
In [9]: e = np.ones((3,2)) # create a array filled with ones
        print("Array of e:\n",e)
```

```
Array of e:
[[1. 1.]
 [1. 1.]
 [1. 1.]
```

```
In [ ]: # Comes under array creation functionality
```

```
In [10]: # create a identity matrix
         f= np.eye(4) # 4X4 identical matrix
         print(" Identity Matric f: \n", f)
```

```
Identity Matric f:
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
```

2. Array Manipulations

```
In [13]: a1 = np.array([1,2,3])
         reshaped = np.reshape(a1,(1,3))# reshape to 1x3
         print("Reshaped array:", reshaped)
```

Reshaped array: [[1 2 3]]

```
In [12]: a1 = np.array([1, 2, 3])
         reshaped = np.reshape(a1, (1, 3)) # reshape to 1x3

         print("Reshaped array:", reshaped)
```

Reshaped array: [[1 2 3]]

```
In [14]: #Flattened Array
         f1 = np.array([[1,2],[3,4],[5,6],[7,8]])
         falttened = np.ravel(f1)
         print("falttened array:", falttened)
```

falttened array: [1 2 3 4 5 6 7 8]

```
In [22]: #Transpose an Array
         e1 = np.array([[1,2],[3,4]])
         transposed = np.transpose(e1)
         print("transposed array:\n", transposed)
```

transposed array:
[[1 3]
[2 4]]

```
In [26]: #Stack array Vertically
         a2 = np.array([1,2])
         b2 = a2 = np.array([3,4])
         stacked = np.vstack([a2,b2])
         print("Stacked array:\n", stacked)
```

Stacked array:
[[3 4]
[3 4]]

```
In [27]: #Stack array Horizontal
         a2 = np.array([1,2])
         b2 = a2 = np.array([3,4])
         stacked = np.hstack([a2,b2])
         print("Stacked array:\n", stacked)
```

Stacked array:
[3 4 3 4]

3 . Mathematical Functions

```
In [28]: # add two array
```

```
g = np.array([1,2,3,4])
added = np.add(g,2) #add 2 to each element
print("Add 2 to g:",added)
```

Add 2 to g: [3 4 5 6]

In [29]: *# add Square array*

```
g = np.array([1,2,3,4])
squared = np.power(g,2) #squared 2 to each element
print("squared to g:",squared)
```

squared to g: [1 4 9 16]

In [36]: *# add Square root array*

```
import numpy as np
g = np.array([1, 2, 3, 4])
sqrt_g = np.sqrt(g) # square root of each element
print("Square root of g: \n", sqrt_g)
```

Square root of g:

[1. 1.41421356 1.73205081 2.]

In [35]: *a3 = np.array([1,2,3,4])*
dot_product = np.dot(a3,g) # dot product work in same dimension of matrix
print("Dot product of a3 X g matrix is ",dot_product)

Dot product of a3 X g matrix is 30

4. Statistical Functions

In [39]: *#Mean*

```
s = np.array([1,2,3,4])
mean = np.mean(s)
print("Mean of s:", mean)
```

Mean of s: 2.5

In [41]: *#Standard Deviation of an array*

```
s = np.array([1,2,3,4])
std_dev = np.std(s)
print("Standard deviation of s:", std_dev)
```

Standard deviation of s: 1.118033988749895

In [42]: *min = np.min(s)*
print("Min of s:", min)

Min of s: 1

In [44]: *max = np.max(s)*
print("Max of s:", max)

Max of s: 4

In []:

In [45]: *#Variance of an array*
s = np.array([1,2,3,4])
var = np.var(s)
print("Variance of var:", var)

Variance of var: 1.25

In [46]: `#Random Sampling`

In [53]: `# Generating random values between 0 and 1
random_vals = np.random.rand(3) # Array of 3 random values between 0 to 1
print("Random Values:", random_vals)`

Random Values: [0.59302255 0.50806296 0.13070795]

In [54]: `np.random.seed(0) # stop generating random numbers for every execution
Generating random values between 0 and 1
random_vals = np.random.rand(3) # Array of 3 random values between 0 to 1
print("Random Values:", random_vals)`

Random Values: [0.5488135 0.71518937 0.60276338]

In [59]: `# Generating random integer values between 0 and 1
np.random.seed(0) # stop generating random numbers for every execution
randoint_vals = np.random.randint(0,10, size =5) # Array of 3 random values betw
print("Random int Values:", randoint_vals)`

Random int Values: [5 0 3 3 7]

7. Boolean and logical fucntion

In [61]: `import numpy as np

a = np.array([1, 2, 3, 4])
b = np.array([2, 2, 2, 2])

result = np.logical_and(a > 1, b == 2)

print("Logical AND:", result)`

Logical AND: [False True True True]

In [62]: `import numpy as np

a = np.array([1, 2, 3, 4])

result = np.logical_or(a < 2, a > 3)

print("Logical OR:", result)`

Logical OR: [True False False True]

In [63]: `import numpy as np

a = np.array([True, False, True])

result = np.logical_not(a)

print("Logical NOT:", result)`

Logical NOT: [False True False]

In [64]: `import numpy as np

a = np.array([True, False, True])
b = np.array([True, True, False])`

```
result = np.logical_xor(a, b)  
print("Logical XOR:", result)
```

Logical XOR: [False True True]

In []:

In []:

In []:

In []:

In []: